Mergesort × Quicksort

Plano da apresentação — divisão do trabalho entre 5 pessoas · 15 minutos

O trabalho: implementar Mergesort e Quicksort em C++, medir o tempo de cada um com vetores de **30, 300, 3.000, 30.000 e 300.000** inteiros aleatórios, **20 execuções por tamanho** (5 × 20 × 2 = **200 ordenações**) e apresentar a comparação em 15 minutos. Já está tudo pronto na pasta `trabalho_ordenacao/`: o código C++, o painel `PAINEL_ADM.bat` (compila e roda tudo; precisa do Dev-C++ ou MinGW) e os slides (`apresentacao/slides.pptx`, com o roteiro de cada um nas anotações). O que falta é estudar a sua parte, colocar os nomes e ensaiar.

Como preencher: escrevam o nome de cada integrante na coluna “Responsável”. Cada pessoa fala ~2,5 a 3 min e prepara os próprios slides. Os tempos já somam 15:00.

Cronograma da apresentação

Tempo	Bloco	Responsável	Slides
0:00 – 2:30	**1. Abertura, problema e metodologia**	Pessoa 1: _____	1 – 3
2:30 – 5:00	**2. Mergesort: como funciona e complexidade**	Pessoa 2: _____	4 – 5
5:00 – 7:30	**3. Quicksort: como funciona e complexidade**	Pessoa 3: _____	6 – 7
7:30 – 10:30	**4. Implementação, medição e demonstração ao vivo**	Pessoa 4: _____	8 – 9
10:30 – 13:30	**5. Resultados, análise e conclusão**	Pessoa 5: _____	10 – 13
13:30 – 15:00	Perguntas (cada um responde sobre a sua parte)	Todos	14

Linha condutora (o que a plateia deve levar para casa)

- Os dois algoritmos são O(n log n) no caso médio, e as medições confirmam: a cada vez que n cresce 10×, o tempo cresce um pouco mais que 10× (o fator log n).
- O Quicksort foi ~15% mais rápido nos tamanhos grandes, porque troca elementos no próprio vetor (melhor uso de cache) e não precisa copiar para um vetor auxiliar.
- O Mergesort ganha em garantia: é O(n log n) mesmo no pior caso e é estável. O Quicksort pode cair para $O(n^2)$ com pivô ruim, e por isso usamos a mediana de três.
- Nos tamanhos pequenos (30 e 300) tudo leva microssegundos e a diferença oscila de uma rodada para outra.

O que cada pessoa faz

Pessoa 1 — Abertura, problema e metodologia

0:00 – 2:30 · slides 1 a 3

O que apresentar

- **Slide 1:** título, nomes do grupo, disciplina.
- **Slide 2:** o problema: por que ordenar importa e por que comparar algoritmos medindo tempo, não só pela notação O.
- **Slide 3:** metodologia: 5 tamanhos, 20 execuções cada, mesmos dados para os dois algoritmos, números aleatórios de 0 a 1.000.000 com semente fixa (42), compilação com `-O2`, máquina usada (processador, sistema operacional).

Antes da apresentação

- Anotar a configuração da máquina onde o teste final for rodado.
- Colocar os nomes do grupo no slide 1.
- Controlar o tempo no ensaio (cada bloco tem horário de início na tabela da página 1).

Pessoa 2 — Mergesort

2:30 – 5:00 · slides 4 e 5

O que apresentar

- **Slide 4:** ideia de dividir e conquistar: divide o vetor ao meio até sobrar 1 elemento e intercala (merge) as metades ordenadas. Mostrar um exemplo com 8 números desenhado passo a passo.
- **Slide 5:** complexidade: O(n log n) no melhor, médio e pior caso; memória extra O(n) (vetor auxiliar, ~1,2 MB para 300.000 inteiros); algoritmo estável.

Antes da apresentação

- Estudar as funções `merge` e `mergesortRec` em `benchmark.cpp`.
- O diagrama do exemplo já está no slide 4: treinar a explicação linha por linha.
- Preparar resposta para: “por que log n níveis?”.

Pessoa 3 — Quicksort

5:00 – 7:30 · slides 6 e 7

O que apresentar

- **Slide 6:** ideia: escolhe um pivô, separa menores à esquerda e maiores à direita (particionamento de Hoare) e repete em cada lado. Exemplo com os mesmos 8 números do slide 4.
- **Slide 7:** complexidade: O(n log n) no caso médio, O(n^2) no pior caso; memória extra só da recursão, O(log n); não é estável. Explicar a mediana de três como defesa contra o pior caso.

Antes da apresentação

- Estudar `quicksortRec` em `benchmark.cpp`.
- O slide 6 usa o mesmo exemplo de 8 números do slide 4: treinar a partição passo a passo.
- Preparar resposta para: “o que acontece com um vetor já ordenado e pivô no primeiro elemento?”.

Pessoa 4 — Implementação, medição e demo

7:30 – 10:30 · slides 8 e 9

O que apresentar

- **Slide 8:** como medimos: `std::chrono::steady_clock` em volta da chamada, uma rodada de aquecimento descartada, os dois algoritmos ordenam cópias do mesmo vetor e `std::is_sorted` confere o resultado. Vetores pequenos: k cópias, tempo dividido por k.
- **Slide 9 / demo:** abrir o `PAINEL_ADM.bat`, escolher a opção [2] (roda em ~2 s) e mostrar o relatório que abre no navegador.

Antes da apresentação

- Testar o `PAINEL_ADM.bat` no computador da apresentação (precisa de Dev-C++ ou MinGW) e levar um print de reserva.
- Rodar a versão final do teste (opções [2] e [3]) e passar a pasta resultados para a Pessoa 5.
- Preparar resposta para: “por que usar a mesma entrada nos dois algoritmos?”.

Pessoa 5 — Resultados, análise e conclusão

10:30 – 13:30 · slides 10 a 13

O que apresentar

- **Slide 10:** tabela de médias e desvios (a da página seguinte).
- **Slide 11:** o gráfico em escala log: as duas retas quase paralelas mostram o mesmo crescimento n log n; o Quicksort fica sempre um pouco abaixo nos tamanhos grandes.
- **Slide 12:** outros cenários de entrada (já ordenado, invertido, muitos repetidos): o Quicksort venceu em todos.
- **Slide 13:** conclusão: quando usar cada um (Quicksort quando velocidade média importa; Mergesort quando precisa de garantia de pior caso ou estabilidade).

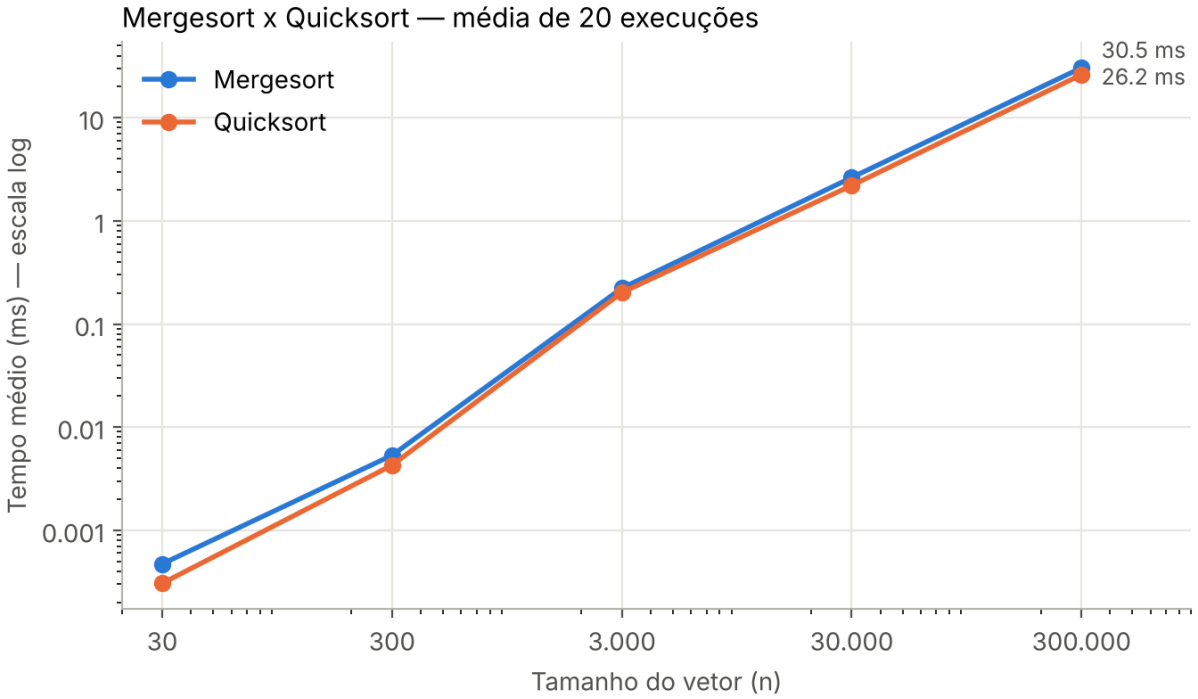
Antes da apresentação

- Se os números mudarem no teste final, atualizar a tabela e os gráficos dos slides 10 a 12.
- Treinar a explicação do crescimento: 10× mais dados → ~12× mais tempo (n log n).
- Conduzir a rodada de perguntas (slide 14).

Resultados já medidos (para os slides 10 e 11)

Média e desvio padrão de 20 execuções, em milissegundos. Medido em um Intel Xeon 2,8 GHz; rodem de novo na máquina de vocês (PAINEL_ADM.bat, opção [2]). Os números mudam, o padrão se mantém.

n	Mergesort (ms)	Quicksort (ms)	Quicksort vs. Mergesort
30	0,0005 ± 0,0001	0,0003 ± 0,0001	35% mais rápido
300	0,0053 ± 0,0006	0,0043 ± 0,0006	20% mais rápido
3.000	0,2226 ± 0,1110	0,2017 ± 0,0803	9% mais rápido
30.000	2,63 ± 0,5695	2,20 ± 0,0480	16% mais rápido
300.000	30,50 ± 1,83	26,16 ± 1,66	14% mais rápido



Leitura rápida: de 30.000 para 300.000 (10× mais dados) o tempo cresceu ~12×, como previsto por n log n (10 × log 300.000 / log 30.000 ≈ 12,2). Um algoritmo O(n²) cresceria 100×.

Checklist do grupo

	Tarefa	Quem
☐	Preencher os nomes na tabela da página 1 e combinar o prazo dos slides	Todos
☐	Nomes colocados no slide 1	Pessoa 1
☐	Teste final rodado, CSVs e gráfico atualizados	Pessoas 4 e 5
☐	Cada um leu as anotações do apresentador dos seus slides	Todos
☐	Ensaio completo cronometrado (meta: 13:30 sem as perguntas)	Todos
☐	Demo testada no computador da apresentação + plano B (print/vídeo)	Pessoa 4